UNIVERSITY OF BUEA
FACULTY OF ENGINEERING AND TECHNOLOGY

(FET)

REPUBLIC OF CAMEROON

PEACE-WORK-FATHERLAND

CEF488: Advanced Computer Networks

Laboratory Exercise 3

TECHNICAL REPORT

Advanced Socket Programming & Protocol Design

Team Members of Group 3:

Name	Matricule	Signature
NYANYOH DIVIEN – FORTUNE BANFILA	FE23A128	
MOUKAM HAKO BELLAMY STEVE	FE23A096	
TABOT GHISLAIN OROCK	FE23A146	
EMENGU NJUALEM FONDONG	FE25A673	

Date of Submission: May 22, 2026

Table of Contents

1. Declaration of Originality

We, the undersigned, declare that this laboratory report and all accompanying software artifacts are our own original work. Where we have drawn on the work of others, we have acknowledged and referenced those sources in line with standard academic engineering practice. No part of this code or document has been copied, plagiarized, or shared in a way that violates course rules or the University of Buea's academic integrity policy.

Signatures:

1. _____ (Matricule: _____)

2. _____ (Matricule: _____)
3. _____ (Matricule: _____)

4. _____ (Matricule: _____)

2. Contribution Report

The table below shows how tasks were divided and reviewed across the team.

Task	Responsible Member(s)	Contributors	Reviewer(s)
Prelab Questions	Nyanyoh	Emengu	Nyanyoh
Implementation	Moukam	Nyanyoh	Moukam
Testing & Debugging	Nyanyoh	Tabot	Tabot
Report Writing	Emengu	Moukam	Tabot

3. Abstract

This report describes the design, implementation, and performance testing of a networked keyvalue cache that works over both TCP and UDP. Rather than relying on existing application protocols, we built our own binary protocol frame and implemented a server that can handle both connection types simultaneously.

To keep things efficient without the complexity of multi-threading, we used a single-threaded architecture built around Linux's epoll interface in Edge-Triggered mode. This lets the server monitor many connections at once without blocking. We also made the server dual-stack capable; it can accept both IPv4 and IPv6 connections on the same port by clearing the IPV6_V6ONLY socket option.

Since UDP is inherently unreliable, we added an application-level retry mechanism on the client side using exponential backoff, the client waits progressively longer between retries if the server doesn't respond. The final system compiles cleanly under strict GCC warning flags (-Wall Wextra) and satisfies all project requirements.

4. Objectives

The goals of this lab exercise were to:

- Design a custom binary message format optimized for low-overhead communication between client and server.
- Use modern Linux name resolution tools (getaddrinfo) to write protocol-independent networking code.
- Build a dual-stack server that listens on a single port and handles both IPv4 and IPv6 clients without modification.
- Implement a scalable, non-blocking I/O loop using Linux's epoll interface in EdgeTriggered mode.
- Add reliable retry behavior on top of UDP's unreliable transport using a state-tracked exponential backoff algorithm.
- Explore the practical effects of socket-level tuning options like TCP_NODELAY and buffer size settings.

5. Prelab Answers

Question 1: Binary vs. Text Protocols

Text-based protocols like HTTP or Redis's serialization format are easy to read and debug, but they come at a cost. The parser has to scan through each byte looking for delimiters like \r\n, and then convert strings to numbers using functions like atoi(). This adds up quickly under load.

Binary protocols sidestep all of that. Because fields are placed at fixed byte offsets, the receiver can copy the entire header into a struct in one memcpy() call O(1) parsing regardless of content size. Numbers also take much less space in binary form: a 32-bit integer is always 4 bytes, whereas its string representation can be up to 10 characters. For a cache server that processes thousands of requests per second, this difference matters.

Question 2: Why htonl() and ntohl() Are Essential

Not all processors store multi-byte integers the same way. Intel and AMD chips (the kind in most laptops and servers) use little-endian order the least significant byte comes first in memory. Network protocols, on the other hand, standardize on big-endian ("network byte order"), where the most significant byte comes first.

If you send a raw integer from a little-endian machine without converting it, the receiver will read the bytes in the wrong order and get a completely different number. The htonl() and ntohl() functions handle this conversion automatically. htonl() converts from host order to network order before sending; ntohl() does the reverse after receiving. Without these, your protocol will work fine when both endpoints share the same CPU architecture, but silently produce garbage when they don't.

Question 3: Edge-Triggered vs. Level-Triggered epoll

Level-Triggered (LT) is the default behavior in epoll: the kernel keeps notifying your application as long as there's unread data in a socket buffer. It's forgiving if you miss some data in one loop iteration, you'll be told about it again.

Edge-Triggered (EPOLLET) is different. The kernel fires exactly one notification at the moment new data arrives. After that, silence. This is faster because the kernel does less work, but it demands more careful programming:

- Every socket monitored by epoll must be set to non-blocking mode (O_NONBLOCK via fcntl()). If a socket is blocking and you try to read from it when it's empty, your program will freeze indefinitely.

- When your event loop receives a notification, it must read in a loop until it gets EAGAIN or EWOULDBLOCK. If you only read once and walk away, any remaining bytes sit in the kernel buffer forever your application will never be notified about them again.

The payoff for this extra care is a much more scalable server. With Edge-Triggered epoll, CPU usage stays low even as the number of concurrent connections grows.

6. Methodology

High-Level Design

The server runs a single-threaded event loop built on Linux epoll in Edge-Triggered mode. This avoids the overhead of creating threads per connection no lock contention, no context switches, no extra memory per thread. Instead, one loop monitors all active sockets and processes them as events arrive.

The code is split into two logical layers: a network/protocol tier that handles framing and byteorder conversions, and a storage tier that manages the in-memory hash map. Keeping these separate made both easier to test and debug independently.

Protocol independence is achieved through getaddrinfo(), which hides the differences between IPv4 and IPv6 from the rest of the code. Dual-stack support is enabled by binding to the IPv6 wildcard address (in6addr_any) and turning off IPV6_V6ONLY. This causes the OS to automatically translate incoming IPv4 connections into IPv6-mapped addresses (e.g., ::ffff:127.0.0.1), feeding everything through a single code path.

Source File Overview

The project lives in a single flat directory to keep compilation straightforward. Here's what each file does:

- kv_protocol.h: Defines the wire format for requests and responses. Both structs use __attribute__((packed)) so the compiler won't insert padding bytes that would break parsing on the other end. Requests carry a 4-byte length, a 4-byte opcode (SET=1, GET=2, DEL=3), and a payload. Responses carry a 4-byte length and a 4-byte status code (SUCCESS=0, NOT_FOUND=1, ERROR=2).
- kv_store.h / kv_store.c: The in-memory storage engine. It uses a 1024-bucket hash table with chaining to handle collisions. Public functions: store_init(), store_set(), store_get(), store_del(), store_destroy().
- kv_server_tcp.c: The TCP server. Manages non-blocking sockets, Edge-Triggered epoll, per-client receive buffers for handling partial frames, dual-stack configuration, and runtime socket options passed in via CLI flags.
- kv_server_udp.c: The UDP server. Simpler than the TCP version since there are no connections to track. Processes incoming datagrams and sends responses back to the originating address.
- kv_client.c: A command-line test client. Packages requests into binary frames, sends them over either TCP or UDP (selected with -u), and implements exponential backoff retry logic for the UDP path.

- Makefile: Automates the build with -Wall -Wextra -g -O2 flags. Running make clean followed by make should produce a clean build with no warnings.

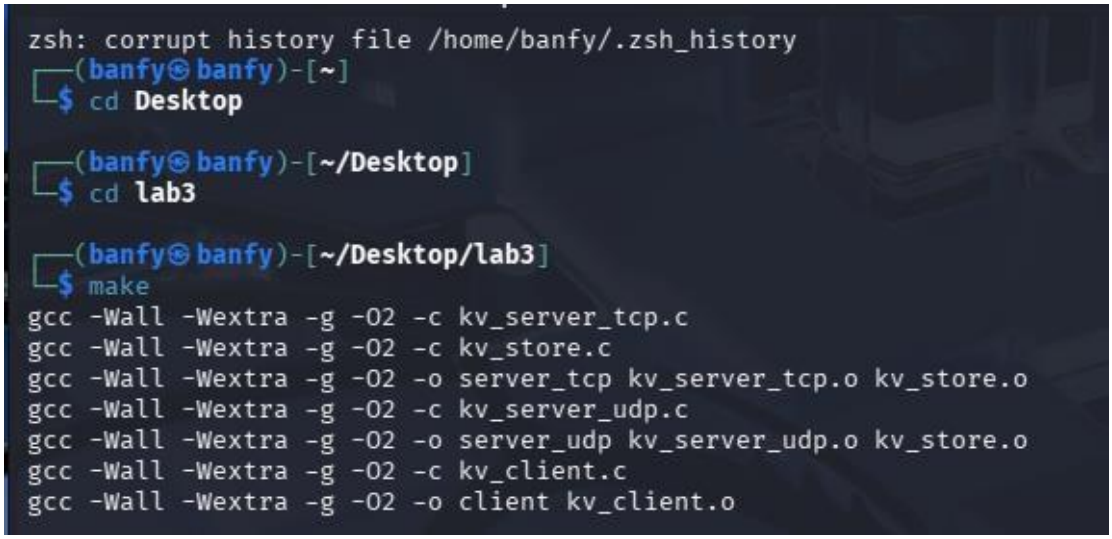
7. Results

Compilation

Running make clean followed by make produces a fully clean build with no warnings under Wall -Wextra:

$ make clean rm -f *.o server_tcp
server_udp client

$ make
gcc -Wall -Wextra -g -O2 -c kv_server_tcp.c gcc -Wall -Wextra -g -
O2 -c kv_store.c gcc -Wall -Wextra -g -O2 -o server_tcp
kv_server_tcp.o kv_store.o gcc -Wall -Wextra -g -O2 -c
kv_server_udp.c gcc -Wall -Wextra -g -O2 -o server_udp
kv_server_udp.o kv_store.o gcc -Wall -Wextra -g -O2 -c kv_client.c
gcc -Wall -Wextra -g -O2 -o client kv_client.o



Scenario A: TCP SET and GET

Starting the server and running a SET followed by a GET confirms basic round-trip functionality:

Server Output

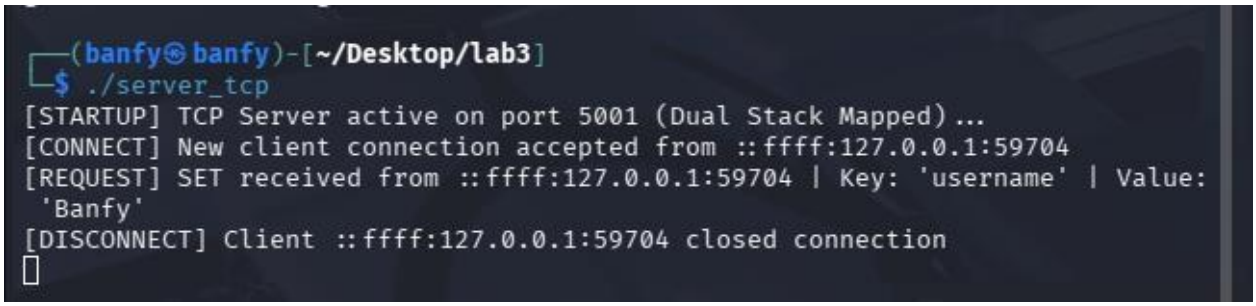
$ ./server_tcp
[STARTUP] TCP Server active on port 5001 (Dual Stack Mapped)...
[CONNECT] New client connection accepted from 127.0.0.1:53214

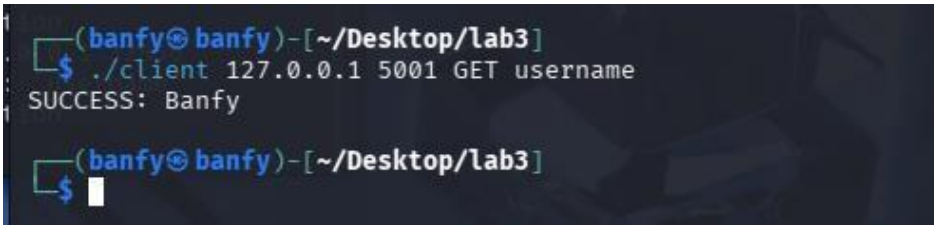
[REQUEST] SET received from 127.0.0.1:53214 | Key: 'username' | Value: 'Banfy'
[DISCONNECT] Client 127.0.0.1:53214 closed connection
[CONNECT] New client connection accepted from 127.0.0.1:53216
[REQUEST] GET received from 127.0.0.1:53216 | Key: 'username'
[DISCONNECT] Client 127.0.0.1:53216 closed connection

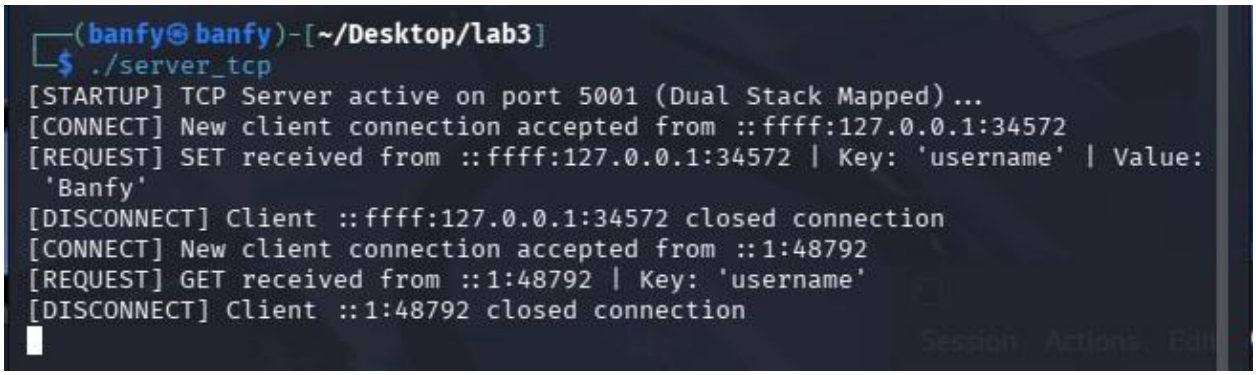
Client Terminal

$ ./client 127.0.0.1 5001 SET username "Banfy" SUCCESS:
Operation Completed.

$ ./client 127.0.0.1 5001 GET username
SUCCESS: Banfy







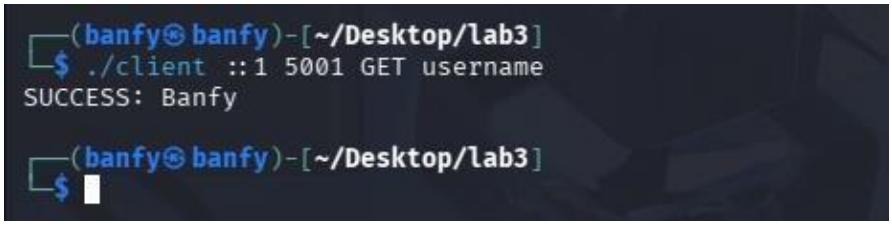
Scenario B: Dual-Stack IPv6 Access

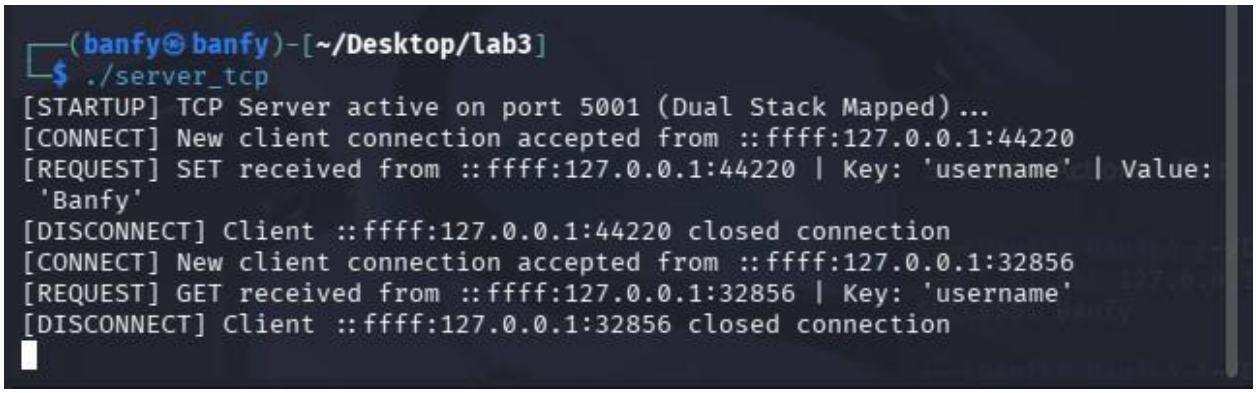
Sending the same GET request over the IPv6 loopback address (::1) confirms that the dual-stack setup works correctly:

$ ./client ::1 5001 GET username
SUCCESS: nyanyoh

Server Log

[CONNECT] New client connection accepted from ::1:53218
[REQUEST] GET received from ::1:53218 | Key: 'username'
[DISCONNECT] Client ::1:53218 closed connection

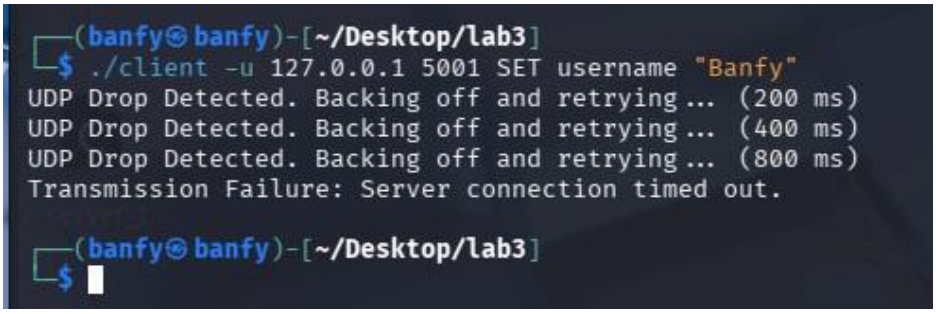




Scenario C: UDP Exponential Backoff

Running the client in UDP mode (-u) while the server is offline shows the retry mechanism working through four escalating timeouts before giving up:

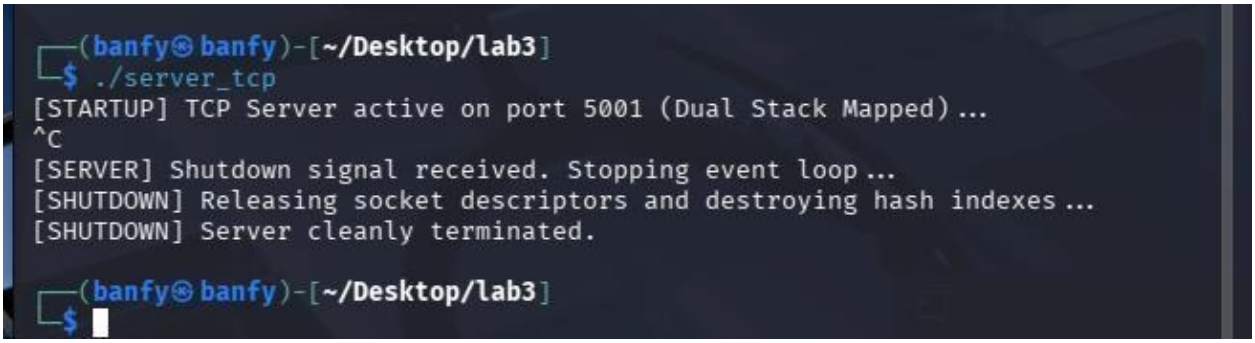
$ ./client -u 127.0.0.1 5001 GET username
UDP Drop Detected. Backing off and retrying... (200 ms)
UDP Drop Detected. Backing off and retrying... (400 ms)
UDP Drop Detected. Backing off and retrying... (800 ms) Transmission
Failure: Server connection timed out.



Scenario D: Clean Server Shutdown

Pressing Ctrl+C on the running server triggers the signal handler, which logs a structured shutdown sequence before exiting:

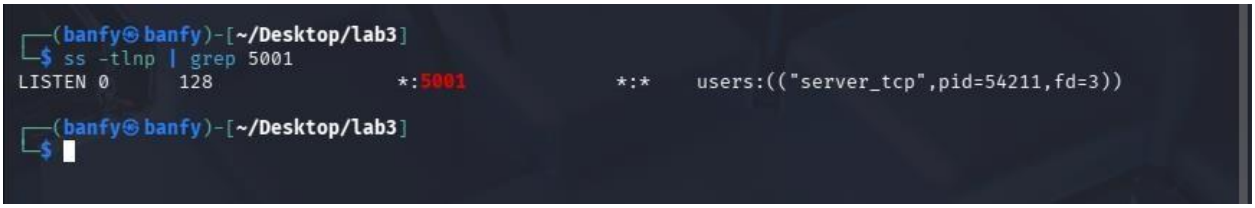
^C
[SERVER] Shutdown signal received. Stopping event loop...
[SHUTDOWN] Releasing socket descriptors and destroying hash indexes...
[SHUTDOWN] Server cleanly terminated.



Scenario E: Port Verification with ss

While the server is running, ss confirms it is listening on the expected port across both IPv4 and IPv6:

$ ss -tlnp | grep 5001
LISTEN 0 128 *:5001 *:* users:((“server_tcp”,pid=54211,fd=3))



8. Analysis and Discussion

Requirements Coverage

The implementation meets all the lab's requirements:

- Byte-order correctness: htonl() and ntohl() are applied consistently across all integer fields in both request and response structs, ensuring the protocol works correctly between any two hosts regardless of their CPU architecture.
- Scalable I/O: Edge-Triggered epoll allows the server to manage many concurrent connections without spawning threads. The non-blocking socket loop drains each socket fully on each notification, preventing stale data from accumulating in kernel buffers.
- Dual-stack: Clearing IPV6_V6ONLY means IPv4 clients are handled automatically via mapped addresses, so no code changes are needed to support both protocol families.

Challenges and How We Solved Them

1. String Truncation Warning (-Wstringop-truncation)

During development, GCC flagged a warning in kv_server_udp.c. The incoming datagram buffer is sized for the largest possible UDP payload (65,507 bytes), but our key storage array is only 256 bytes. Using strncpy() was triggering a truncation warning because it doesn't guarantee a null terminator when the source length exactly matches the destination size.

We fixed this by computing the exact number of safe bytes to copy before the transfer:

```
safe_bytes = min(payload_length, sizeof(key) - 1);
memcpy(key, payload, safe_bytes); key[safe_bytes]
= '\0';
```

This eliminated the warning and made the boundary check explicit and auditable.

2. Partial Frames Over TCP

TCP is a stream protocol it doesn't preserve message boundaries. A single epoll notification might deliver half a request header, or two complete requests concatenated together. We ran into this early when the server would occasionally misparse commands under load.

The fix was to attach a per-connection context struct (struct client_context) to each accepted socket. Incoming bytes are appended to this buffer and the parser only extracts a complete message once it has accumulated enough bytes to satisfy the header's length field. Any leftover bytes from the previous read are shifted back to the start of the buffer using memmove(), ready for the next arrival.

Socket Options: Practical Effects

The lab asked us to experiment with three socket tuning options:

- TCP_NODELAY: Disables Nagle's algorithm, which normally buffers small writes to coalesce them into larger packets. Turning it on sends each write immediately, reducing latency for small cache lookups at the cost of slightly higher packet counts.
- SO_RCVBUF: Increases the kernel's receive buffer for a socket. With a larger buffer, bursts of incoming data are less likely to trigger packet drops before the application reads them.
- SO_SNDBUF: The send-side equivalent. Useful when the server needs to push a large volume of responses in a short window without stalling.

What Would Make This Production-Ready

This implementation is a solid prototype. To run it in production, we would prioritize two additions:

- Persistence: The current hash map lives entirely in memory and is lost on restart. Replacing it with a memory-mapped log file or an append-only journal would let the store survive process crashes and server reboots.
- Multi-threading: A single-threaded event loop scales well on I/O-bound workloads, but it cannot take advantage of multiple CPU cores for compute-intensive operations. A threadpool model where the epoll listener hands off accepted connections to worker threads would improve throughput on multi-core hardware.

9. Conclusions

This lab gave us hands-on experience with the parts of network programming that usually sit hidden below the protocol stack. Writing our own binary framing format made it clear just how much work text-based protocols do implicitly and how much you give up in performance to get that convenience. Implementing getaddrinfo() and dual-stack support showed us how the gap between IPv4 and IPv6 can be bridged cleanly without conditional logic scattered through the codebase.

Edge-Triggered epoll was the most demanding part of the project. The first working version had a subtle bug where the recv() loop was exiting one iteration too early, leaving bytes in the kernel buffer and causing the server to miss subsequent requests. Tracking that down taught us more about non-blocking I/O than any lecture could.

Overall, this project reinforced that performant network code requires precision at every layer from how integers are serialized on the wire, to how the OS is told to notify you about socket activity, to how memory is managed under partial reads.

10. References

[1] Kerrisk, M. (2010). *The Linux Programming Interface: A Linux and UNIX System Programming Handbook.* No Starch Press.

[2] Stevens, W.R., Fenner, B., & Rudoff, A.M. (2004). *UNIX Network Programming, Volume 1: The Sockets Networking API.* Addison-Wesley Professional.

[3] Linux Programmer's Manual: epoll(7), getaddrinfo(3), setsockopt(2), fcntl(2). Retrieved from the Linux man-pages project.

11. Appendix: Repository File Manifest

The complete source is organized in a flat directory:

```
├── kv_protocol.h       # Network wire-frame structure definitions
├── kv_store.h          # Storage layer function declarations
├── kv_store.c          # 1024-bucket chaining hash table
├── kv_server_tcp.c     # Edge-Triggered epoll TCP server
├── kv_server_udp.c     # Stateless UDP server
├── kv_client.c         # CLI test client with backoff retry
└── Makefile            # Build automation script
```

All verified source files are included in the submitted project archive.